Końcowa dokumentacja projektu

System wyznaczania współrzędnych dla wizualizacji grafów planarnych

Implementacja w języku C

Autorzy:
Krzysztof Wasilewski, Jakub Pietrzkiewicz

Data: 14.04.2026

Spis treści

1 Streszczenie ... 4
2 Wprowadzenie ... 5
2.1 Cel projektu ... 5
2.2 Zakres dokumentu ... 5
3 Charakterystyka funkcjonalna aplikacji ... 6
3.1 Funkcje udostępniane użytkownikowi ... 6
3.2 Interfejs uruchomieniowy CLI ... 6
3.3 Przykładowe scenariusze użycia ... 6
4 Architektura oprogramowania ... 7
4.1 Podział na moduły ... 7
4.2 Przepływ sterowania ... 7
5 Model danych ... 8
5.1 Struktury podstawowe ... 8
5.2 Struktury pomocnicze ... 8
5.3 Kody statusu ... 8
6 Specyfikacja wejścia i wyjścia ... 9
6.1 Wejście tekstowe ... 9
6.2 Mechanizm dwuprzebiegowy odczytu ... 9
6.3 Wyjście tekstowe ... 9
6.4 Wyjście binarne ... 9
7 Szczegółowy opis kodu ... 10
7.1 Moduł `main` ... 10
7.2 Moduł `graph` ... 10
7.3 Moduł `fruchterman` ... 10
7.4 Moduł `adjacency_list` ... 11
7.5 Moduł `tutte` ... 11
7.6 Moduł `utils` ... 11
8 Opis algorytmiczny ... 12
8.1 Fruchterman-Reingold ... 12
8.2 Tutte-like embedding ... 12
9 Zarządzanie pamięcią i odporność ... 14
9.1 Strategia alokacji ... 14
9.2 Strategia cleanup ... 14
9.3 Potencjalne punkty ryzyka pamięci ... 14
10 Obsługa błędów ... 15
10.1 Kategorie błędów ... 15
10.2 Kody powrotu i znaczenie ... 15
10.3 Komunikaty diagnostyczne ... 15
11 Proces budowania i uruchamiania ... 16
11.1 Makefile ... 16
11.2 Instrukcja uruchomienia ... 16
12 Testowanie i weryfikacja ... 17
12.1 Zakres testów jednostkowych ... 17
12.2 Wynik aktualnego uruchomienia ... 17
12.3 Ograniczenia algorytmiczne ... 17
12.4 Ograniczenia implementacyjne ... 17
12.5 Ryzyka eksploatacyjne ... 17
13 Szczegółowa analiza funkcji i założeń ... 18
13.1 `main(int argc, char *argv[])` ... 18

13.2 `load_graph(char path[], Graph **graph_out)` ... 19
13.3 `save_graph_as_text(Graph *graph, char path[])` ... 20
13.4 `save_graph_as_binary(Graph *graph, char path[])` ... 21
13.5 `run_fruchterman(...)` ... 21
13.6 `create_adjacency_list(...)` i `free_adjacency_list(...)` ... 22
13.7 `run_tutte(...)` ... 23
13.8 Funkcje pomocnicze `utils` ... 24
14 Scenariusze operacyjne i diagnostyka ... 26
14.1 S1: Poprawne uruchomienie domyślne ... 26
14.2 S2: Nieistniejący plik wejściowy ... 26
14.3 S3: Błędny format danych ... 26
14.4 S4: Nieznana opcja ... 26
14.5 S5: Niepoprawna temperatura ... 26
14.6 S6: Graf bez krawędzi przy Fruchterman ... 27
14.7 S7: Małe grafy przy Tutte ... 27
15 Podsumowanie ... 28
16 Załącznik A: Skrócona referencja API (C) ... 29
16.1 graph.h ... 29
16.2 fruchterman.h ... 29
16.3 tutte.h ... 29
16.4 adjacency_list.h ... 29
16.5 utils.h ... 29
17 Załącznik B: Przykładowe dane ... 30
17.1 Przykładowy plik wejściowy ... 30
17.2 Przykładowe uruchomienie ... 30
17.3 Przykładowy wynik tekstowy ... 30

1 Streszczenie

Projekt realizuje aplikację konsolową w języku C, która dla grafu planarnego zapisanego jako lista krawędzi wyznacza współrzędne wierzchołków przeznaczone do dalszej wizualizacji. Program implementuje dwa algorytmy:

- Fruchterman-Reingold (algorytm siłowy),
- Tutte embedding (osadzenie z relaksacją sąsiadów i ustalonym brzegiem).

Aplikacja obsługuje uruchamianie parametryzowane z linii poleceń, wejście tekstowe oraz dwa formaty wyjścia (tekstowy i binarny). Kod ma budowę modularną, pokrycie testami jednostkowymi i jednolity system kodów powrotu. Dokument przedstawia:

- szczegółową analizę kodu,
- opis architektury i przepływu sterowania,
- formaty danych,
- ocenę zgodności z wymaganiami,
- ograniczenia i propozycje rozwoju.

2 Wprowadzenie

2.1 Cel projektu

Celem projektu jest dostarczenie narzędzia CLI, które przyjmuje graf opisany jako lista krawędzi i generuje współrzędne 2D wierzchołków, aby możliwa była czytelna i estetyczna wizualizacja struktury grafu. Kluczowe założenia:

- prosty i przewidywalny interfejs uruchomieniowy,
- możliwość wyboru algorytmu rozmieszczenia,
- eksport wyników do formatu tekstowego lub binarnego,
- przenośna i testowalna implementacja C.

2.2 Zakres dokumentu

Niniejsza dokumentacja jest dokumentacją końcową projektu C i stanowi integrację informacji z dokumentacji funkcjonalnej oraz implementacyjnej. Zawiera opis dla:

- użytkownika technicznego (jak uruchomić, jakie dane podać, jak interpretować wynik),
- programisty utrzymującego (jak działa kod, gdzie szukać logiki, jakie są kontrakty funkcji),
- zespołu integrującego (jak czytać pliki wyjściowe i jak mapować je do kolejnych komponentów, np. Java GUI).

3 Charakterystyka funkcjonalna aplikacji

3.1 Funkcje udostępniane użytkownikowi

Program udostępnia funkcje:

- odczyt grafu z pliku tekstowego,
- uruchomienie wybranego algorytmu layoutu,
- konfigurację liczby iteracji oraz parametrów przestrzeni roboczej,
- zapis wyniku do formatu tekstowego,
- zapis wyniku do formatu binarnego.

3.2 Interfejs uruchomieniowy CLI

Składnia ogólna:

```
./graph [opcje] <input_file> <output_file>
```

Opcje:

- `-a`, `--algorithm <fruchterman|tutte>`
- `-i`, `--iterations <int>`
- `-t`, `--temperature <double>`
- `-s`, `--size <double>`
- `-f`, `--format <text|binary>`
- `-h`, `--help`

Wartości domyślne:

- `algorithm = fruchterman`
- `iterations = 1000`
- `temperature = 10.0`
- `size = 1000.0`
- `format = text`

3.3 Przykładowe scenariusze użycia

Domyślne uruchomienie:

```
./graph dane.txt wynik.txt
```

Uruchomienie algorytmu Tutte:

```
./graph -a tutte dane.txt wynik.txt
```

Eksport binarny:

```
./graph -f binary dane.txt wynik.bin
```

Uruchomienie z jawną konfiguracją:

```
./graph -a fruchterman -i 2500 -t 12.5 -s 1400 -f text dane.txt wynik.txt
```

4 Architektura oprogramowania

4.1 Podział na moduły

Kod źródłowy jest podzielony na 6 modułów logicznych:

- `main` - punkt wejściowy programu i orkiestracja przepływu,
- `graph` - odczyt i zapis grafu, mapowanie identyfikatorów,
- `fruchterman` - algorytm siłowy,
- `tutte` - algorytm osadzenia Tutte,
- `adjacency_list` - pomocnicza reprezentacja sąsiadów,
- `utils` - wspólne kody statusu i struktura listy jednokierunkowej.

4.2 Przepływ sterowania

Przebieg pojedynczego uruchomienia:

1. `main` parsuje argumenty i waliduje parametry.
2. `load_graph` odczytuje plik i buduje strukturę `Graph`.
3. `main` wybiera algorytm (`run_fruchterman` albo `run_tutte`).
4. Wynik jest zapisywany przez `save_graph_as_text` lub `save_graph_as_binary`.
5. Pamięć jest zwalniana przez `free_graph`.
6. Program zwraca kod statusu.

5 Model danych

5.1 Struktury podstawowe

Model grafu oparty jest o tablice dynamiczne:

- `Node`:
  - `id` - identyfikator logiczny,
  - `x`, `y` - współrzędne.
- `Edge`:
  - `first_node_index`, `second_node_index` - indeksy do tablicy `nodes`,
  - `weight` - waga krawędzi,
  - `label[33]` - etykieta (do 32 znaków + terminator).
- `Graph`:
  - `nodes`, `nodes_count`,
  - `edges`, `edges_count`.

5.2 Struktury pomocnicze

W module `adjacency_list`:

- `Neighbor` - element listy sąsiadów (`node_index`, `weight`, `next`),
- `AdjacencyList` - tablica list sąsiadów i tablica stopni wierzchołków.

W module `utils`:

- `List` - prosta lista jednokierunkowa używana przy pierwszym skanie pliku do budowy zbioru unikalnych ID.

5.3 Kody statusu

System kodów powrotu (enum):

- `SUCCESS = 0`
- `ARGUMENTS_ERROR = 1`
- `FILE_ERROR = 2`
- `INPUT_FORMAT_ERROR = 3`
- `ALGORITHM_ERROR = 4`
- `MEMORY_ERROR = 5`

6 Specyfikacja wejścia i wyjścia

6.1 Wejście tekstowe

Kazda linia danych ma postac:

```
<label> <nodeA> <nodeB> <weight>
```

Przykład:

```
AB 1 2 1.0
BC 2 3 0.8
CA 3 1 1.2
```

Zasady parsowania:

- parser akceptuje białe znaki (`space`, `tab`, `CR`, `LF`),
- parser ignoruje całe linie komentarza zaczynające się od `#`,
- parser toleruje komentarze inline po rekordzie,
- etykieta jest ograniczona do 32 znakow (`%32s`).

6.2 Mechanizm dwuprzebiegowy odczytu

Odczyt grafu realizowany jest dwuprzebiegowo:

1. Przebieg pierwszy:
   - zliczenie krawędzi,
   - zbudowanie zbioru unikalnych identyfikatorów wierzchołków.
2. Przebieg drugi:
   - mapowanie identyfikatorów na indeksy,
   - wypełnienie tablicy `edges`.

Korzyść: ograniczenie reallocacji i precyzyjna alokacja pamięci.

6.3 Wyjście tekstowe

Format:

```
<node_id> <x> <y>
```

Przykład:

```
1 0.000000 0.000000
2 1000.000000 0.000000
3 500.000000 866.000000
```

6.4 Wyjście binarne

Dla każdego wierzchołka zapisywany jest rekord 20-bajtowy:

- 4 bajty: `int id`,
- 8 bajtów: `double x`,
- 8 bajtów: `double y`.

Uwagi integracyjne:

- plik nie zawiera nagłówka,
- kolejność rekordów odpowiada kolejności w tablicy `nodes` (po sortowaniu po `id`),

7 Szczegółowy opis kodu

7.1 Moduł `main`

Rola:

- inicjalizacja parametrów domyślnych,
- parsowanie opcji CLI,
- walidacja danych sterujących,
- wybór algorytmu i formatu zapisu,
- jednolita obsługa kodów wyjścia.

Najważniejsze decyzje implementacyjne:

- wymaganie obecności dwóch argumentów pozycyjnych (`input_file`, `output_file`),
- walidacja `iterations > 0`, `temperature > 0`, `size > 0`,
- jawna walidacja nazw algorytmu i formatu.

Opis parsera argumentów:

- parser przechodzi po `argv` liniowo,
- opcje wymagające parametru zużywają kolejny token,
- `-h/--help` kończy program kodem sukcesu,
- nieznany token skutkuje `ARGUMENTS_ERROR`.

7.2 Moduł `graph`

Rola:

- odczyt grafu z pliku,
- mapowanie ID wierzchołków na indeksy tablicowe,
- zapis wyników,
- zwalnianie pamięci struktury `Graph`.

Szczegóły implementacyjne:

- dane wierzchołków są sortowane funkcją `qsort`,
- mapowanie ID realizowane przez `bsearch` (`get_node_index`),
- podczas odczytu używane jest pomocnicze pomijanie komentarzy i białych znaków.

Mocne strony:

- dobra wydajność mapowania ID (wyszukiwanie binarne),
- jasny podział na odczyt i zapis,
- ograniczenie rozmiaru etykiety zabezpiecza bufor.

7.3 Moduł `fruchterman`

Rola:

- implementacja iteracyjnego algorytmu siłowego.

Fazy obliczen:

1. Losowa inicjalizacja pozycji w kwadracie `[0,size] x [0,size]`.
2. Obliczenie sił odpychania dla każdej pary wierzchołków.
3. Obliczenie sił przyciągania dla każdej krawędzi.
4. Aktualizacja pozycji z ograniczeniem kroku przez temperature.
5. Chłodzenie liniowe temperatury.

Stabilizacja numeryczna:

- w odległości dodawane jest `1e-9`, aby uniknąć dzielenia przez zero.

Uwagi praktyczne:

- wynik nie jest deterministyczny przez `srand(time(NULL))`,
- dla grafów bez krawędzi funkcja zwraca `ARGUMENTS_ERROR`.

7.4 Moduł `adjacency_list`

Rola:

- konwersja tablicy krawędzi grafu do listy sąsiedztwa,
- policzenie stopni wierzchołków,
- zwolnienie pamięci pomocniczej.

Szczegóły:

- każda krawędź dodawana dwukierunkowo (graf nieskierowany),
- `degrees[i]` jest inkrementowane dla obu końców krawędzi,
- alokacja przez `calloc` zeruje wskaźniki i liczniki.

7.5 Moduł `tutte`

Rola:

- implementacja osadzenia Tutte z automatycznym wyznaczeniem ramy.

Przebieg:

1. Dla `nodes_count < 4` uruchamiana jest procedura specjalna.
2. Dla większych grafów tworzona jest lista sąsiedztwa.
3. Proba znalezienia cyklu brzegowego (`find_cycle_perimeter`).
4. Jeśli brak sukcesu, fallback do 4 wierzchołków o najwyższych stopniach (`find_cycle_backup`).
5. Rozmieszczenie ramy na obwodzie kwadratu.
6. Inicjalizacja wewnętrznych wierzchołków w centrum.
7. Iteracyjna relaksacja do zbieżności (`epsilon = 0.0001`) lub limitu iteracji.

7.6 Moduł `utils`

Rola:

- definicja kodów statusu,
- mini-biblioteka listy jednokierunkowej (`list_contains`, `list_prepend`, `list_free`).

Znaczenie:

- jeden punkt definicji kodów błędów,
- prosty komponent wspierający odczyt grafu bez zależności zewnętrznych.

8 Opis algorytmiczny

8.1 Fruchterman-Reingold

Model matematyczny:

- optymalna odległość:

$$k = \sqrt{\frac{A}{|V|}}, \quad A = s^2$$

gdzie s oznacza bok obszaru roboczego (`size`).

- siła odpychania:

$$f_{r(d)} = \frac{k^2}{d}$$

- siła przyciągania z wagą:

$$f_{a(d)} = w \cdot \frac{d^2}{k}$$

Złożoność obliczeniowa:

- odpychanie: $O(|V|^2)$ na iteracje,
- przyciąganie: $O(|E|)$ na iteracje,
- całkowicie: $O((|V|^2 + |E|) \cdot I)$.

Właściwości:

- dobrze działa dla szerokiej klasy grafów,
- kosztowne dla dużych grafów,
- podatne na minima lokalne.

8.2 Tutte-like embedding

Idea:

- część wierzchołków jest unieruchomiona na brzegu,
- pozostałe przyjmują średnią ważoną położeń sąsiadów.

Równania relaksacji:

$$x_i = \frac{\sum_{j \in N(i)} w_{ij} x_j}{\sum_{j \in N(i)} w_{ij}}$$

$$y_i = \frac{\sum_{j \in N(i)} w_{ij} y_j}{\sum_{j \in N(i)} w_{ij}}$$

Warunek stopu:

- `max_displacement < epsilon` lub
- przekroczenie limitu `max_iterations`.

Właściwości:

- duża stabilność dla grafów średniej wielkości,

- deterministyczność przy danym grafie
- jakość zależy od trafności wyboru ramy.

9 Zarządzanie pamięcią i odporność

9.1 Strategia alokacji

Stosowane są:

- `malloc` dla obiektów i tablic,
- `calloc` dla tablic wymagających zerowania,
- kontrola null po każdej alokacji.

9.2 Strategia cleanup

W przypadku błędu:

- funkcje zwalniają to, co zostało już zaalokowane,
- zwracany jest adekwatny kod błędu,
- wyższa warstwa (`main`) dokańcza cleanup obiektu `Graph`.

9.3 Potencjalne punkty ryzyka pamięci

- przy bardzo dużych grafach koszt pamięci list sąsiedztwa i tablic tymczasowych może być wysoki,
- brak limitów na liczbę rekordów pliku może prowadzić do wyczerpania zasobów systemowych.

10 Obsługa błędów

10.1 Kategorie błędów

Program obsługuje:

- błędy argumentów i konfiguracji,
- błędy operacji plikowych,
- błędy formatu danych,
- błędy algorytmiczne,
- błędy alokacji pamięci.

10.2 Kody powrotu i znaczenie

- `0` - sukces,
- `1` - błąd argumentów,
- `2` - błąd plikowy,
- `3` - niepoprawny format wejściowy,
- `4` - błąd algorytmu,
- `5` - błąd pamięci.

10.3 Komunikaty diagnostyczne

Najważniejsze komunikaty wyświetlane na `stderr`:

- `Error: Unknown option ...`
- `Error: Input and output files must be specified.`
- `Error: Invalid iteration value`
- `Error: Invalid temperature value`
- `Error: Invalid size value`
- `Error: Cannot open input file: ...`
- `Error: Graph must have at least one node and one edge.`
- `Error: Memory allocation failed`

11 Proces budowania i uruchamiania

11.1 Makefile

Cele:

- `make` - budowa `bin/graph`,
- `make test` - budowa i uruchomienie testów,
- `make clean` - usunięcie `bin`.

Flagi kompilacji:

- `-Wall`
- `-lm`

Uwagi:

- linkowanie z `-lm` zapewnia funkcje matematyczne (`sqrt`, `fmin`),
- testy linkują się z obiektami projektu poza `main.o`.

11.2 Instrukcja uruchomienia

Kompilacja:

```
make
```

Uruchomienie testów:

```
make test
```

Uruchomienie programu:

```
./bin/graph [opcje] input.txt output.txt
```

12 Testowanie i weryfikacja

12.1 Zakres testów jednostkowych

Zestaw testów pokrywa:

- odczyt grafu i mapowanie ID (`test_graph.c`),
- obsługę komentarzy i błędnego formatu,
- tworzenie list sasiedztwa i stopnie (`test_adjacency_list.c`),
- zachowanie `run_fruchterman` dla grafu pustego i małego,
- zachowanie `run_tutte` dla grafu małego i grafu pełnego.

12.2 Wynik aktualnego uruchomienia

W bieżącej wersji projektu testy przechodzą poprawnie (`All tests passed!`).

12.3 Ograniczenia algorytmiczne

- Fruchterman-Reingold ma koszt kwadratowy względem liczby wierzchołków.
- Jakość layoutu zależy od parametrów (`iterations`, `temperature`, `size`).
- Tutte-like embedding zależy od heurystycznego wyboru ramy.

12.4 Ograniczenia implementacyjne

- Brak parsera z „twardą” diagnostyką numerów linii (kod `line_number` jest utrzymywany, ale nie jest eksponowany w komunikatach).
- Brak wsparcia dla danych binarnych na wejściu (tylko tekst).
- Brak opcji wymuszenia ziarna RNG w Fruchterman (utrudniona reprodukowalność).

12.5 Ryzyka eksploatacyjne

- Dla bardzo dużych grafów czas i pamięć mogą być nieakceptowalne bez optymalizacji.
- Niepoprawne dane (np. semantycznie niespójny graf) mogą prowadzić do layoutu o niskiej jakości bez jednoznacznego błędu.

13 Szczegółowa analiza funkcji i założeń

13.1 `main(int argc, char *argv[])`

Wejście:

- `argc`, `argv` z systemu operacyjnego.

Wyjście:

- kod procesu zgodny z enum statusów.

Założenia:

- funkcja oczekuje dwóch argumentów pozycyjnych i opcjonalnych flag,
- dla opcji wymagających argumentu oczekuje poprawnego tokenu po opcji,
- po poprawnej konfiguracji wywołuje dokładnie jeden algorytm i jeden zapis.

Sytuacje wyjątkowe:

- nieznana opcja,
- brak plików,
- niepoprawne wartości `iterations`, `temperature`, `size`,
- nieznany algorytm lub format.

Kod funkcji (fragmenty):

```
if (input_file == NULL || output_file == NULL) {
  fprintf(stderr, "Error: Input and output files must be specified.\n");
  return ARGUMENTS_ERROR;
}

for (int i = 1; i < argc; i++) {
  if (strcmp(argv[i], "-h") == 0 || strcmp(argv[i], "--help") == 0) {
    printf("Usage: ./graph [options] <input_file> <output_file>\n");
    return 0;
  }
  if ((strcmp(argv[i], "-a") == 0 || strcmp(argv[i], "--algorithm") == 0) && i + 1 <
argc) {
    algorithm = argv[++i];
  } else if ((strcmp(argv[i], "-f") == 0 || strcmp(argv[i], "--format") == 0) && i + 1 <
argc) {
    format = argv[++i];
  }
}

if (strcmp(algorithm, "fruchterman") == 0) {
  status = run_fruchterman(graph, temperature, iterations, size);
} else if (strcmp(algorithm, "tutte") == 0) {
  status = run_tutte(graph, size, iterations);
} else {
  fprintf(stderr,
          "Error: Unknown algorithm '%s'. Use 'fruchterman' or 'tutte'.\n",
          algorithm);
  free_graph(graph);
  return ARGUMENTS_ERROR;
}

if (strcmp(format, "text") == 0) {
  status = save_graph_as_text(graph, output_file);
```

```
} else if (strcmp(format, "binary") == 0) {
  status = save_graph_as_binary(graph, output_file);
} else {
  fprintf(stderr, "Error: Unknown format '%s'.\n", format);
  free_graph(graph);
  return ARGUMENTS_ERROR;
}

free_graph(graph);
return status;
```

13.2 `load_graph(char path[], Graph **graph_out)`

Wejście:

- ścieżka do pliku tekstowego.

Wyjście:

- wskaźnik do zaalokowanej struktury `Graph`.

Założenia:

- plik musi być czytelny,
- każdy rekord danych musi mieć 4 pola (`label int int double`),
- komentarze rozpoczynają się od `#`.

Inwarianty po sukcesie:

- `nodes_count >= 0`, `edges_count >= 0`,
- tablica `nodes` jest posortowana rosnąco po `id`,
- każda krawędź ma poprawne indeksy do tablicy `nodes`.

Złożoność:

- skan 1: $O(E \cdot U)$ dla budowy zbioru unikalnych ID na liście (gdzie `U` to liczba unikalnych wierzchołków),
- sortowanie: $O(V \log V)$,
- skan 2: $O(E \log V)$ przez `bsearch`.

Kod funkcji (fragmenty):

```
int result = fscanf(file, "%32s %d %d %lf", label, &first_node,
                    &second_node, &weight);

if (result == 4) {
  edges_count++;
  int ids[2] = {first_node, second_node};
  for (int i = 0; i < 2; i++) {
    if (!list_contains(unique_nodes, ids[i])) {
      list_prepend(&unique_nodes, ids[i]);
      nodes_count++;
    }
  }
} else if (result != EOF) {
  list_free(unique_nodes);
  fclose(file);
  return INPUT_FORMAT_ERROR;
}

rewind(file);
edge_index = 0;
while ((result = fscanf(file, "%32s %d %d %lf", label, &first_node,
```

```
                &second_node, &weight)) == 4) {
  int node_index = get_node_index(graph, first_node);
  int node_index_2 = get_node_index(graph, second_node);
  if (node_index < 0 || node_index_2 < 0) {
    free_graph(graph);
    fclose(file);
    return INPUT_FORMAT_ERROR;
  }
  graph->edges[edge_index].first_node_index = node_index;
  graph->edges[edge_index].second_node_index = node_index_2;
  graph->edges[edge_index].weight = weight;
  strncpy(graph->edges[edge_index].label, label, sizeof(graph->edges[edge_index].label)
- 1);
  graph->edges[edge_index].label[sizeof(graph->edges[edge_index].label) - 1] = '\0';
  edge_index++;
}

if (result != EOF) {
  list_free(unique_nodes);
  fclose(file);
  return INPUT_FORMAT_ERROR;
}

qsort(graph->nodes, graph->nodes_count, sizeof(Node), compare_nodes_by_id);

for (int i = 0; i < graph->edges_count; i++) {
  graph->edges[i].first_node_index = get_node_index(graph, graph-
>edges[i].first_node_index);
  graph->edges[i].second_node_index = get_node_index(graph, graph-
>edges[i].second_node_index);
  if (graph->edges[i].first_node_index < 0 || graph->edges[i].second_node_index < 0) {
    free_graph(graph);
    fclose(file);
    return INPUT_FORMAT_ERROR;
  }
}
```

13.3 `save_graph_as_text(Graph *graph, char path[])`

Założenia:

- dla każdego wierzchołka zapisuje jeden rekord `id x y`,
- brak nagłówka i metadanych,
- dokładność liczbowa zgodna z `%lf`.

Uwagi:

- format jest najprostszy do debugowania i ręcznej walidacji,
- może być większy rozmiarowo od binarnego przy dużych grafach.

Kod funkcji (fragment):

```
for (int i = 0; i < graph->nodes_count; i++) {
  if (fprintf(file, "%d %lf %lf\n", graph->nodes[i].id,
        graph->nodes[i].x, graph->nodes[i].y) < 0) {
    fclose(file);
    return FILE_ERROR;
  }
}
```

```
if (fflush(file) != 0) {
  fclose(file);
  return FILE_ERROR;
}
```

13.4 `save_graph_as_binary(Graph *graph, char path[])`

Założenia:

- każdy rekord ma 20 bajtów,
- dane są zapisywane sekwencyjnie bez separatorów.

Uwagi integracyjne:

- format jest szybki w odczycie, ale mniej odporny na niezgodności platformowe,
- w praktyce warto dodać nagłówek wersji formatu.

Kod funkcji (fragment):

```
for (int i = 0; i < graph->nodes_count; i++) {
  if (fwrite(&graph->nodes[i].id, sizeof(int), 1, file) != 1 ||
    fwrite(&graph->nodes[i].x, sizeof(double), 1, file) != 1 ||
    fwrite(&graph->nodes[i].y, sizeof(double), 1, file) != 1) {
    fclose(file);
    return FILE_ERROR;
  }
}

if (fclose(file) != 0)
  return FILE_ERROR;
```

13.5 `run_fruchterman(...)`

Warunki wejściowe:

- `graph != NULL`,
- liczba iteracji dodatnia (walidowana wyżej),
- `size > 0`.

Warunki wyjściowe:

- pozycje wierzchołków znajdują się w granicach `[0,size]`,
- tablice pomocnicze są zwolnione.

Uwagi o jakości wyniku:

- dla grafów rzadkich przy niskiej temperaturze możliwe „zamrożenie” układów,
- dla grafów gęstych i dużych wag może występować silne ściskanie klastrów.

Kod funkcji (fragmenty):

```
for (int i = 0; i < liczba_w; i++) {
  graph->nodes[i].x = random_double(0.0, size);
  graph->nodes[i].y = random_double(0.0, size);
}
for (int a = 0; a < liczba_w; a++) {
  for (int b = 0; b < liczba_w; b++) {
    if (a != b) {
      double dx = graph->nodes[a].x - graph->nodes[b].x;
      double dy = graph->nodes[a].y - graph->nodes[b].y;
      double odleglosc = sqrt(dx * dx + dy * dy) + 1e-9;
      double fr = (k * k) / odleglosc;
```

```
      sila_x[a] += (dx / odleglosc) * fr;
      sila_y[a] += (dy / odleglosc) * fr;
    }
  }
}
for (int i = 0; i < liczba_w; i++) {
  double speed = sqrt(sila_x[i] * sila_x[i] + sila_y[i] * sila_y[i]);
  if (speed > 0.0) {
    double step = fmin(speed, temperature);
    graph->nodes[i].x += (sila_x[i] / speed) * step;
    graph->nodes[i].y += (sila_y[i] / speed) * step;
  }
  if (graph->nodes[i].x < 0.0)
    graph->nodes[i].x = 0.0;
  if (graph->nodes[i].x > size)
    graph->nodes[i].x = size;
  if (graph->nodes[i].y < 0.0)
    graph->nodes[i].y = 0.0;
  if (graph->nodes[i].y > size)
    graph->nodes[i].y = size;
}

temperature *= 0.95;
```

13.6 `create_adjacency_list(...)` i `free_adjacency_list(...)`

for (int i = 0; i < graph->nodes_count; i++) { graph->nodes[i].x = fmin(fmax(graph->nodes[i].x, 0.0), size); graph->nodes[i].y = fmin(fmax(graph->nodes[i].y, 0.0), size); }

Założenia tworzenia:

- dla każdej krawędzi nieskierowanej dodawane są dwa wpisy sąsiedztwa,
- `degrees[i]` odzwierciedla stopień i-tego wierzchołka.

Założenia zwalniania:

- wszystkie elementy list sąsiedztwa oraz tablice pomocnicze są zwalniane,
- funkcja toleruje `NULL`.

Kod funkcji (fragment):

```
if (add_neighbor(&adj_list->adjacency_list[first_index], second_index,
                 weight) != SUCCESS ||
    add_neighbor(&adj_list->adjacency_list[second_index], first_index,
                 weight) != SUCCESS) {
  free_adjacency_list(adj_list);
  fprintf(stderr, "Error: Memory allocation failed\n");
  return MEMORY_ERROR;
}
adj_list->degrees[first_index]++;
adj_list->degrees[second_index]++;

for (int edge = 0; edge < graph->edges_count; edge++) {
  int a = graph->edges[edge].first_node_index;
  int b = graph->edges[edge].second_node_index;
  Neighbor *forward = malloc(sizeof(Neighbor));
  Neighbor *backward = malloc(sizeof(Neighbor));
  if (forward == NULL || backward == NULL)
    return MEMORY_ERROR;
  forward->node_index = b;
```

```
  forward->weight = graph->edges[edge].weight;
  forward->next = adj_list->adjacency_list[a];
  adj_list->adjacency_list[a] = forward;
  backward->node_index = a;
  backward->weight = graph->edges[edge].weight;
  backward->next = adj_list->adjacency_list[b];
  adj_list->adjacency_list[b] = backward;
  adj_list->degrees[a]++;
  adj_list->degrees[b]++;
}

for (int node = 0; node < adj_list->size; node++) {
  Neighbor *current = adj_list->adjacency_list[node];
  while (current != NULL) {
    current = current->next;
  }
}
```

13.7 `run_tutte(...)`

Warunki wejściowe:

- graf musi być zaalokowany,
- `size > 0`, `max_iterations > 0`.

Warunki wyjściowe:

- punkty ramy pozostają na obwodzie kwadratu,
- wierzchołki wewnętrzne są aktualizowane relaksacyjnie,
- pozycje pozostają w obrębie kwadratu.

Uwagi merytoryczne:

- implementacja nie wyznacza formalnie planarności ani prawdziwej zewnętrznej ściany,
- przy fallbacku 4 najwyższych stopni uzyskany layout może odbiegać od klasycznego embeddingu Tutte.

Kod funkcji (fragmenty):

```
static int find_cycle(AdjacencyList *adj_list, int **cycle, int *cycle_len) {
  return (find_cycle_perimeter(adj_list, cycle, cycle_len) == SUCCESS)
            ? SUCCESS
            : find_cycle_backup(adj_list, cycle, cycle_len);
}
```

```
Neighbor *neighbor = adj_list->adjacency_list[i];
while (neighbor != NULL) {
  int neighbor_idx = neighbor->node_index;
  double weight = neighbor->weight;

  sum_weighted_x += weight * graph->nodes[neighbor_idx].x;
  sum_weighted_y += weight * graph->nodes[neighbor_idx].y;

if (temperature < 0.01)
  break;
  sum_weights += weight;

  neighbor = neighbor->next;
}

if (sum_weights > 0.0) {
  double new_x = sum_weighted_x / sum_weights;
```

```
  double new_y = sum_weighted_y / sum_weights;
  graph->nodes[i].x = new_x;
  graph->nodes[i].y = new_y;
}

for (int iteration = 0; iteration < max_iterations; iteration++) {
  double max_displacement = 0.0;
  for (int i = 0; i < graph->nodes_count; i++) {
    if (is_boundary_node[i])
      continue;
    /* ... relaksacja ... */
  }
  if (max_displacement < epsilon)
    break;
}

for (int i = 0; i < graph->nodes_count; i++) {
  if (is_boundary_node[i]) {
    graph->nodes[i].x = boundary_x[i];
    graph->nodes[i].y = boundary_y[i];
  }
}
```

13.8 Funkcje pomocnicze `utils`

`list_contains`, `list_prepend`, `list_free` tworzą minimalny, celowy zestaw narzędzi.

Silne strony:

- prostota i przewidywalność,
- brak ukrytych efektów ubocznych.

Słabości:

- przy bardzo dużych danych koszt wyszukiwania liniowego w `list_contains` może być istotny.

Kod funkcji (fragment):

```
int list_contains(List *head, int value) {
  for (List *cur = head; cur != NULL; cur = cur->next)
    if (cur->data == value)
      return 1;
  return 0;
}

int list_prepend(List **head, int value) {
  List *node = malloc(sizeof(List));
  if (node == NULL)
    return MEMORY_ERROR;
  node->data = value;
  node->next = *head;
  *head = node;
  return SUCCESS;
}

List *list_clone(List *head) {
  List *copy = NULL;
  List *tail = NULL;
  for (List *current = head; current != NULL; current = current->next) {
    List *node = malloc(sizeof(List));
    if (node == NULL) {
      list_free(copy);
```

```
      return NULL;
    }
    node->data = current->data;
    node->next = NULL;
    if (copy == NULL) {
      copy = node;
      tail = node;
    } else {
      tail->next = node;
      tail = node;
    }
  }
  return copy;
}

void list_free(List *head) {
  while (head != NULL) {
    List *next = head->next;
    free(head);
    head = next;
  }
}
```

14 Scenariusze operacyjne i diagnostyka

14.1 S1: Poprawne uruchomienie domyślne

Polecenie:

```
./bin/graph graph.txt output.txt
```

Oczekiwane zachowanie:

- odczyt pliku,
- uruchomienie Fruchterman-Reingold,
- zapis `output.txt`,
- kod powrotu `0`.

Objawy sukcesu:

- istnieje plik wyjściowy,
- liczba rekordów wyjściowych odpowiada liczbie unikalnych ID.

14.2 S2: Nieistniejący plik wejściowy

Polecenie:

```
./bin/graph brak.txt out.txt
```

Oczekiwany rezultat:

- komunikat `Error: Cannot open input file: ...`,
- kod powrotu `2`.

14.3 S3: Błędny format danych

Przykładowy rekord błędny:

```
AB 1 2 NIE_LICZBA
```

Oczekiwany rezultat:

- kod `3`,
- brak poprawnie wygenerowanego pliku wynikowego.

14.4 S4: Nieznana opcja

Polecenie:

```
./bin/graph --algoX tutte in.txt out.txt
```

Oczekiwany rezultat:

- komunikat o nieznanej opcji,
- kod `1`.

14.5 S5: Niepoprawna temperatura

Polecenie:

```
./bin/graph -t 0 in.txt out.txt
```

Oczekiwany rezultat:

- komunikat `Error: Invalid temperature value`,
- kod `1`.

14.6 S6: Graf bez krawędzi przy Fruchterman

Zachowanie:

- funkcja algorytmu zwraca błąd argumentu,
- brak zapisu wyniku,
- program kończy się kodem `1`.

14.7 S7: Małe grafy przy Tutte

Zachowanie:

- `n=1`: pojedynczy punkt w centrum,
- `n=2`: dwa punkty po przekątnej kwadratu,
- `n=3`: trójkąt o pozycjach specjalnych.

Znaczenie:

- zapewnia sensowny layout bazowy i brak problemów numerycznych dla skrajnie małych wejść.

15 Podsumowanie

Projekt C realizuje wymagany trzon funkcjonalny: odczyt danych grafu, dwa algorytmy wyznaczania layoutu, eksport wyników i obsługę uruchomienia z linii poleceń. Implementacja jest modularna, czytelna i testowalna. Niniejsza dokumentacja końcowa łączy perspektywę użytkową i implementacyjną, opisuje kod szczegółowo oraz wskazuje rzeczywiste granice systemu.

W tej postaci projekt jest gotowy do przekazania dalej jako silnik obliczeniowy i baza dla kolejnego etapu (warstwa wizualna i interakcyjna).

16 Załącznik A: Skrócona referencja API (C)

16.1 graph.h

- `int load_graph(char path[], Graph **graph_out)`
- `int save_graph_as_binary(Graph *graph, char path[])`
- `int save_graph_as_text(Graph *graph, char path[])`
- `int free_graph(Graph *graph)`
- `int get_node_index(Graph *graph, int node_id)`

16.2 fruchterman.h

- `int run_fruchterman(Graph *graph, double initial_temperature, int max_iterations, double size)`

16.3 tutte.h

- `int run_tutte(Graph *graph, double size, int max_iterations)`

16.4 adjacency_list.h

- `int create_adjacency_list(Graph *graph, AdjacencyList **adj_list_out)`
- `void free_adjacency_list(AdjacencyList *adj_list)`

16.5 utils.h

- `int list_contains(List *head, int value)`
- `int list_prepend(List **head, int value)`
- `void list_free(List *head)`

17 Załącznik B: Przykładowe dane

17.1 Przykładowy plik wejściowy

```
# Prostokąt z przekątną
AB 1 2 1.0
BC 2 3 1.0
CD 3 4 1.0
DA 4 1 1.0
AC 1 3 1.2
```

17.2 Przykładowe uruchomienie

```
./bin/graph -a tutte -i 1200 -s 1000 -f text input.txt output.txt
```

17.3 Przykładowy wynik tekstowy

```
1 0.000000 0.000000
2 1000.000000 0.000000
3 1000.000000 1000.000000
4 0.000000 1000.000000
```

Zestaw powyzszych przykladow pokazuje przekroj calego systemu: od wejÅcia i walidacji, przez rdzen algorytmiczny, po testy i cleanup pamieci. Sekcja moze sluzyc jako praktyczny „atlas kodu” dla osob rozwijajacych projekt.